

RAPPORT TECHNIQUE COMPLET (ENVI. 3000 MOTS)

Projet: Retrieval Engine Competition (Kaggle) - Dossier val

1) Resume executif

Ce rapport presente une vue technique complete du projet present dans le dossier val. Le projet porte sur la recherche d'information et la classification de requetes a partir d'un grand corpus de documents multi-categorie. Il s'appuie sur un dataset Kaggle nomme retrieval-engine-competition, avec 216,041 documents et un jeu de requetes d'entrainement de 327 questions, ainsi qu'un jeu de test de 141 requetes. Les documents et requetes appartiennent a cinq categories principales: android, gaming, programmers, tex, unix. Le dossier contient des notebooks pour l'analyse exploratoire (EDA), l'indexation TF-IDF, l'indexation BM25+, l'encodage par Sentence Transformers, et la consolidation des resultats de baselines. Il contient aussi des artefacts de modeles (vectoriseurs, index, embeddings) et des sorties de visualisation. Enfin, un script de classification predict_category.py permet d'entrainer un classifieur de categories (TF-IDF + regression logistique) et d'alimenter un fichier de soumission.

Les principaux enseignements techniques sont les suivants:

- Les methodes lexicales (TF-IDF, BM25+) fournissent une baseline interpretable mais limitent la recuperation semantique.
- Les embeddings denses (SentenceTransformers all-MiniLM-L6-v2) obtiennent les meilleurs scores sur Recall@10, Precision@10 et MRR.
- La separation des categories dans l'espace d'embeddings est visible via t-SNE et UMAP, ce qui soutient l'usage d'un classifieur de categories dans une phase de reranking.
- Le pipeline de soumission combine la recherche (top-100 documents) et la classification de categorie des requetes.

Ce rapport s'appuie uniquement sur les elements du dossier val, en particulier les notebooks et les fichiers de sortie.

2) Contexte et objectifs

Le projet vise a construire un moteur de recherche d'information sur un corpus massif de documents techniques. Le probleme se decompose en deux taches principales:

1. Retrieval: pour une requete donnee, retrouver une liste de documents pertinents (top-k).
2. Classification: predire la categorie associee a la requete.

Le dataset Kaggle fournit pour l'entrainement des requetes et leurs documents pertinents (ground truth), et demande en soumission un format qui combine liste de documents recuperes et categorie. L'objectif du projet est donc d'etablir des baselines solides pour le retrieval, puis de disposer d'un classifieur de categories stable. Le dossier val montre une Phase 1 axee retrieval (TF-IDF, BM25+, SentenceTransformers), puis une Phase 2 qui integre la classification dans la generation du fichier de soumission.

3) Structure du dossier et artefacts

Le dossier val est organise de maniere classique pour un projet de data science:

- data/
 - * docs.json: corpus de documents (id, title, text, tags, category)
 - * queries_train.json: requetes d'entrainement (id, text, title, tags, category)
 - * qgts_train.json: ground truth (mapping query_id -> documents pertinents)
 - * queries_test.json: requetes de test (sans label)
 - * df_docs_preprocessed.pkl: corpus pretraite (content + content_clean)

- * submission.csv: gabarit de soumission (top-k + categorie)
- models/
 - * tfidf_vectorizer.pkl, tfidf_matrix.pkl: artefacts TF-IDF
 - * bm25_index.pkl: index BM25+
 - * corpus_embeddings.npy: embeddings du corpus (Sentence Transformers)
- notebooks/
 - * 01_eda.ipynb: exploration des donnees et pretraitement
 - * 02_tfidf.ipynb: modele TF-IDF et evaluation
 - * 03_bm25.ipynb: modele BM25+ et evaluation
 - * 04_embeddings.ipynb: Sentence Transformers, recherche semantique, visualisations t-SNE/UMAP
 - * 05_baseline_results.ipynb: consolidation des resultats et generation de la soumission
- outputs/
 - * images: distributions et visualisations (category_distribution.png, document_length_distribution.png, embedding_viz_tsne.png, embedding_viz_umap.png, baseline_comparison.png, query_top_words.png, top_words.png)
 - * resultats.pkl: tfidf_results.pkl, bm25_results.pkl, embeddings_results.pkl
 - * submission.csv: soumission Kaggle (top-100)
 - * autres fichiers CSV d exemple
- predict_category.py
Script autonome de classification des requetes.

4) Donnees et schema

Le corpus contient 216,041 documents, chacun avec les champs id, text, title, tags, category. Les requetes d entrainement sont 327 et partagent les memes champs (sauf que leur category est la cible), tandis que les requetes de test sont 141 et servent uniquement a la soumission. Le ground truth est fournie via qgts_train.json, qui mappe chaque query_id vers une liste de documents pertinents, avec un champ total_relevant_docs et des paires (doc_id, relevance).

Le notebook 01_eda.ipynb confirme:

- 216,041 documents
- 5 categories: android, gaming, programmers, tex, unix
- 327 requetes train, 141 requetes test
- longueur moyenne des documents (apres nettoyage) ~83.8 tokens
- nombre moyen de documents pertinents par requete ~9.1
- longueur moyenne des requetes (train) ~11.4 mots, min 3, max 31

Le dataset est donc fortement desbalance en termes de taille: peu de requetes pour un corpus massif, ce qui rend l evaluation sensible au choix de metriques et rend le retrieval plus complexe. La taille du corpus impose aussi des choix de performances (batching, indexation, cache d embeddings).

5) Pretraitement et construction du contenu

Le notebook 01_eda.ipynb cree un champ content par concatenation de title, text et tags. Cette representation unifie les informations disponibles dans un champ textuel unique, facilite les vectorisations, et permet aux methodes lexicales de tirer parti des tags. Un pretraitement content_clean est ensuite applique:

- conversion en minuscules
- suppression de la ponctuation
- suppression des chiffres isoles
- tokenisation via NLTK
- suppression des stopwords anglais

- suppression des tokens courts ($\text{len} \leq 2$)

Le resultat est sauvegarde sous `data/df_docs_preprocessed.pkl` pour reutilisation dans les autres notebooks. Ce pretraitement est central pour TF-IDF et BM25+, qui sont sensibles au bruit lexical. Pour les embeddings, le corpus brut (content) est conserve afin de profiter de la richesse semantique du texte original.

6) Analyse exploratoire (EDA)

L EDA vise a comprendre la distribution des categories, la longueur des documents et la nature des requetes. Les graphiques produits montrent:

- Distribution des categories (`category_distribution.png`): l ensemble des documents est reparti sur cinq categories avec des volumes differents. Cette distribution est importante pour evaluer les biais potentiels dans le retrieval et la classification.
- Distribution des longueurs de documents (`document_length_distribution.png`): l histogramme indique une longueur moyenne d environ 84 tokens apres nettoyage, avec une variance notable selon les categories. La longueur impacte directement les scores TF-IDF et BM25, ce qui justifie l usage de BM25+ pour une normalisation plus fine.
- Nuage de mots / top words (`wordcloud_corpus.png` ou `top_words.png`): les termes les plus frequents du corpus sont extraits a partir d un echantillon de 5000 documents pour limiter l usage memoire. Cette visualisation permet de verifier que le pretraitement isole bien les termes informatifs.
- Analyse des requetes (`query_top_words.png`): top 15 mots des requetes d entraînement, utiles pour comprendre le vocabulaire typique des utilisateurs.

Ces analyses servent aussi d entree pour les choix de modeles, en particulier le choix de `ngram_range` et de `stop_words` pour TF-IDF, et le choix d embeddings semantiques pour capturer les paraphrases.

7) Modeles de retrieval: TF-IDF

Le notebook `02_tfidf.ipynb` construit un index TF-IDF sur le corpus `content_clean`. Parametres cles:

- `ngram_range=(1,2)`: unigrammes + bigrammes
- `min_df=1`, `max_df=0.95`
- `sublinear_tf=True` pour attenuer l impact des mots trop frequents

Le TF-IDF produit une matrice sparse de dimension (216041, 6,495,428) environ, ce qui souligne la haute dimensionalite et la sparsity du vocabulaire. La recherche se fait via similarite cosinus entre le vecteur de requete et la matrice du corpus. La fonction `search(query,k)` renvoie les indices top-k et leurs scores.

Evaluation TF-IDF (K=10, 327 requetes):

- `Recall@10`: 0.1052
- `Precision@10`: 0.0615
- `MRR`: 0.1514

Ces valeurs reflektent une baseline lexicale: les resultats sont corrects quand les termes exacts apparaissent dans les documents pertinents, mais le modele est limite pour les paraphrases et la synonymie. Les artefacts `tfidf_vectorizer.pkl` et `tfidf_matrix.pkl` sont stockes dans `models/`.

8) Modeles de retrieval: BM25+

Le notebook `03_bm25.ipynb` construit un index BM25+ a partir du corpus

tokenise (content_clean). BM25+ est une extension de BM25 qui ajoute un parametre delta, garantissant un score non nul pour les termes presents. Parametres par default (rank_bm25): k1=1.5, b=0.75, delta=0.5.

Le vocabulaire recense environ 426,388 termes, et la longueur moyenne des documents est ~83.8 tokens (coherent avec l'EDA). La recherche se fait via bm25.get_scores sur les tokens de la requete.

Evaluation BM25+ (K=10):

- Recall@10: 0.1215
- Precision@10: 0.0743
- MRR: 0.2034

BM25+ surpasse TF-IDF sur toutes les metriques. La comparaison montre un gain notable en MRR (0.2034 vs 0.1514). L'index BM25+ est sauvegarde dans models/bm25_index.pkl, ce qui facilite sa reutilisation pour un reranking ou une phase ulterieure.

9) Modeles de retrieval: Sentence Transformers

Le notebook 04_embeddings.ipynb met en place une recherche semantique a partir d'embeddings denses generes par un modele pre-entraîne SentenceTransformer all-MiniLM-L6-v2. Les points essentiels:

- Dimension des embeddings: 384
- Encodage du corpus avec batching (batch_size=32)
- Normalisation des embeddings pour utiliser la similarite cosinus via un simple produit scalaire
- Cache des embeddings dans models/corpus_embeddings.npy pour eviter le recalcul.

La fonction search(query,k) encode la requete et calcule les similarites avec l'ensemble du corpus. Cette methode est plus couteuse en calcul que les approches lexicales mais capte les relations semantiques.

Evaluation Sentence Transformers (K=10):

- Recall@10: 0.1959
- Precision@10: 0.1141
- MRR: 0.2745

Ces scores sont nettement superieurs a TF-IDF et BM25+. Cela confirme que la similarite semantique apporte une valeur forte pour ce type de questions techniques, en particulier quand la requete ne partage pas de termes exacts avec les documents pertinents.

10) Metriques d'evaluation

Les metriques utilisees dans tous les notebooks de retrieval sont:

- Recall@k: fraction des documents pertinents retrouves dans le top-k.
- Precision@k: fraction du top-k qui est pertinente.
- MRR (Mean Reciprocal Rank): inverse du rang du premier document pertinent, moyen sur l'ensemble des requetes.

Ces metriques sont pertinentes pour un moteur de recherche car elles mesurent a la fois la couverture (recall) et la qualite en tete de liste (precision et MRR). Dans ce contexte, la MRR est un signal important car elle reflète la position du premier document pertinent, crucial pour l'experience utilisateur.

11) Comparaison globale des baselines

Le notebook 05_baseline_results.ipynb consolide les resultats des trois approches et genere un tableau comparatif. Les scores sont:

- TF-IDF: Recall@10 0.1052, Precision@10 0.0615, MRR 0.1514
- BM25+: Recall@10 0.1215, Precision@10 0.0743, MRR 0.2034
- SentenceTransformers: Recall@10 0.1959, Precision@10 0.1141, MRR 0.2745

Sentence Transformers domine toutes les metriques. BM25+ est une amelioration systematique par rapport a TF-IDF, surtout en MRR. Le graphique `baseline_comparison.png` visualise ces differences et met en evidence l ecart significatif entre les approches lexicales et semantiques.

12) Analyse qualitative

Les notebooks TF-IDF, BM25+ et `baseline_results` montrent des exemples de requetes pour lesquelles les modeles reussissent ou echouent. Trois types de requetes sont etudiees: facile, ambigue, difficile.

- Cas facile: requetes avec termes explicites et concordance forte avec les documents du corpus. Les trois modeles retrouvent des documents proches du sujet, mais peuvent manquer les documents pertinents exacts en top-5 selon la formulation.
- Cas ambigue: requetes combinant plusieurs themes (ex: web-development, python, sql). Les modeles lexicaux montrent des resultats incoherents, souvent polarises par des termes dominants (ex: SQL). Sentence Transformers se comporte mieux en termes de semantique globale mais reste sensible aux signaux dominants.
- Cas difficile: requetes paraphrasees ou tres specifiques (ex: tikz formatting). Les modeles lexicaux echouent souvent a retrouver des documents pertinents, tandis que Sentence Transformers remonte des documents semantiquement proches, mais pas toujours exacts.

Cette analyse conforte l idee que les embeddings sont necessaires pour traiter la variabilite linguistique, mais aussi que la precision peut encore etre amelioree via reranking ou hybridation (lexical + semantique).

13) Visualisation des embeddings (t-SNE / UMAP)

Le notebook `04_embeddings.ipynb` fournit une exploration visuelle de l espace d embeddings. Deux methodes de reduction dimensionnelle sont utilisees:

- t-SNE sur un echantillon de 3000 documents. Cette reduction montre des clusters qui se regroupent par categorie. Cela suggere que les embeddings capturent bien la structure semantique globale.
- UMAP sur l ensemble du corpus. UMAP offre une separation des categories plus globale et plus interpretable, au prix d un cout de calcul plus eleve. Les clusters par categorie sont visibles, avec des zones de recouvrement qui indiquent des proximites semantiques.

Ces visualisations sont importantes pour justifier la phase suivante: un classifieur de categories peut exploiter cette separabilite pour aider au reranking et au filtrage.

14) Classification des requetes: `predict_category.py`

Le script `predict_category.py` entraine un classifieur TF-IDF + LogisticRegression sur les requetes d entraînement. Le pipeline est:

1. Construction du texte de requete (text + title + tags).
2. Vectorisation TF-IDF (ngrams 1-2, min_df=2, max_df=0.9, stop_words='english').
3. Entraînement d un LogisticRegression (max_iter=2000, multi_class auto).
4. Split train/validation (80/20, stratified) avec mesure accuracy et

- macro F1.
- 5. Entraînement final sur toutes les requêtes train et prédiction des catégories des requêtes test.
- 6. Mise à jour du fichier outputs/Team_92i_ElMorjen.csv.

Ce script fournit une brique stable et reproductible pour la classification. La validation avec accuracy et macro F1 donne un retour rapide sur la qualité, tandis que le fit final sur tous les exemples exploite au mieux le faible nombre de requêtes disponibles.

15) Génération de la soumission Kaggle (Phase 2)

Le notebook 05_baseline_results.ipynb va plus loin en générant une soumission complète. Il sélectionne le meilleur modèle retrieval selon la MRR (SentenceTransformers), puis produit pour chaque requête de test une liste top-100 de documents (K_SUBMIT=100). En parallèle, il entraîne un classifieur de catégories (TF-IDF + LogisticRegression) sur les requêtes train et prédit la catégorie de chaque requête test. Le fichier submission.csv combine:

- query_id
- relevant_doc_ids (liste top-100)
- category

Cette intégration illustre un pipeline complet retrieval + classification, conforme aux exigences du concours Kaggle. Les artefacts de sortie sont conservés dans outputs/.

16) Discussion sur les choix techniques

Plusieurs choix sont pertinents et cohérents avec les contraintes du dataset:

- Le prétraitement lexical est important pour TF-IDF et BM25+, mais les embeddings utilisent le texte brut afin de conserver la sémantique.
- L'usage de ngram_range=(1,2) permet de capturer des expressions courtes utiles dans les questions techniques.
- BM25+ est plus robuste que TF-IDF à la longueur des documents, ce qui est important sur un corpus hétérogène.
- SentenceTransformers all-MiniLM-L6-v2 est un compromis efficace entre performance et coût computationnel. Le caching d'embeddings est indispensable pour un corpus de 216k documents.
- L'évaluation sur un petit nombre de requêtes (327) implique une variance potentiellement élevée. La MRR est donc un indicateur utile mais doit être interprété avec prudence.

17) Limites et risques

Le projet présente plusieurs limites structurelles:

- Taille réduite du jeu de requêtes: seulement 327 requêtes train, ce qui limite la robustesse du classifieur et la précision des métriques.
- Pas de validation croisée: les évaluations sont faites sur l'ensemble des requêtes train, ce qui ne mesure pas toujours la généralisation.
- Approche retrieval purement dense ou purement lexicale: la combinaison hybride (lexical + dense) n'est pas encore explorée.
- Coût de calcul: l'encodage des embeddings est coûteux. Sans GPU, le temps d'exécution peut être long, et certaines réductions (UMAP) sont lourdes sur l'ensemble du corpus.

- Qualite du ground truth: le nombre de documents pertinents par requete est limite, ce qui peut rendre la precision tres sensible a de petites variations.

18) Pistes d amelioration

Plusieurs axes d amelioration sont possibles a partir de cette base:

- Hybridation: combiner BM25+ et SentenceTransformers (fusion de scores) pour exploiter la precision lexicale et la generalisation semantique.
- Reranking: appliquer un reranker neural (cross-encoder) sur le top-100 pour ordonner plus finement les candidats.
- Enrichissement des requetes: reformuler les requetes via expansion (synonymes, tags inferes) pour ameliorer recall.
- Optimisation du classifieur: essayer des modeles plus puissants (SVM, linear SVC, ou fine-tuning) et integrer des features derivees du retrieval (scores, categories des documents recuperes).
- Evaluation plus robuste: utiliser une validation croisee ou un hold-out plus strict pour comparer les baselines de facon stable.

19) Reproductibilite et execution

Les notebooks sont auto-suffisants si les donnees Kaggle sont placees dans data/. L EDA genere df_docs_preprocessed.pkl, qui est reutilise dans les notebooks TF-IDF et BM25+. Les embeddings sont caches dans models/. Les scripts utilisent des chemins relatifs dans le dossier notebooks/.

Execution indicative:

- Lancer 01_eda.ipynb pour generer df_docs_preprocessed.pkl et les plots.
- Lancer 02_tfidf.ipynb, 03_bm25.ipynb pour les baselines lexicales.
- Lancer 04_embeddings.ipynb pour les embeddings et les visualisations.
- Lancer 05_baseline_results.ipynb pour la consolidation et la soumission.
- Pour la classification seule: python predict_category.py.

20) Conclusion

Le dossier val constitue une base solide et bien structuree pour un projet de retrieval et de classification. Les notebooks permettent une progression logique: EDA, baselines lexicales, embeddings semantiques, consolidation des resultats et generation de la soumission. Les resultats montrent clairement l avantage des approches semantiques (Sentence Transformers) sur les methodes classiques (TF-IDF, BM25+). Le projet met egalement en place un classifieur de categories exploitable, avec un script autonome et une integration dans la soumission.

Les artefacts generes (models, outputs, visualisations) facilitent la reproductibilite et le diagnostic. Les limitations principales sont liees a la taille reduite des requetes train et aux couts computationnels des embeddings. Les pistes d amelioration (hybridation, reranking, expansion de requetes) sont naturelles et pourraient conduire a des gains substantiels.

Ce rapport est integralement derive du contenu du dossier val et fournit une vision technique exhaustive, conforme a un format academique.

21) Annexe technique (details operatoires)

Cette annexe synthetise quelques points pratiques issus des notebooks qui peuvent etre utiles pour l operationnalisation. Les calculs TF-IDF et BM25+ utilisent un mapping id_to_idx pour relier les doc_id aux indices de la matrice, ce qui simplifie le calcul des metriques (intersection

entre retrieved_indices et relevant_ids). Le TF-IDF construit une matrice sparse tres large; il est recommande de conserver le vectorizer et la matrice sous forme pickle afin d eviter un recalcul couteux. Dans les notebooks, ces fichiers sont stockes dans models/ et recharges dans 05_baseline_results.ipynb pour l analyse comparative.

Pour les embeddings, le cache corpus_embeddings.npy est essentiel; sans ce cache, l encodage du corpus complet (216k documents) est long et peut dépasser plusieurs dizaines de minutes selon la machine. Le notebook indique l usage de normalize_embeddings=True, ce qui permet un produit scalaire direct pour la similarite cosinus. La visualisation t-SNE est effectuee sur un echantillon (3000 documents) pour eviter un cout trop eleve, alors que la visualisation UMAP est appliquee sur l ensemble du corpus quand la bibliotheque est disponible.

Enfin, la generation de la soumission (top-100) respecte le format query_id, relevant_doc_ids, category, avec relevant_doc_ids encode en JSON pour conserver la liste ordonnee. Cette representation permet une compatibilite directe avec les attentes du concours. Pour un usage production, une etape de monitoring serait utile (latence, taille des index, taux de cache, distribution des scores), ainsi qu une politique de re-entrainement ou de recalcul periodique si le corpus evolue.